Week 2: Inputs, Expressions, and Decisions

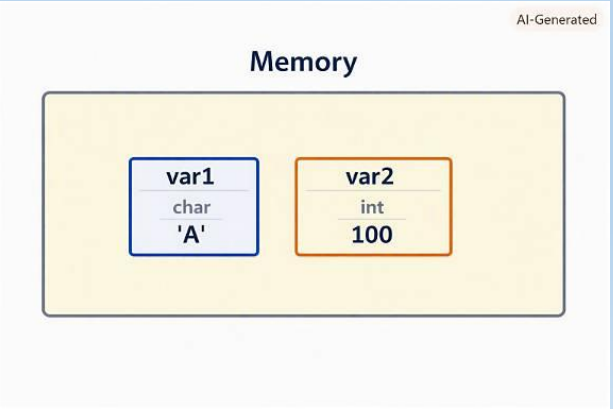
ID1063 • Introduction to Programming in C

Instructor: Rakesh Venkat

Recap from week 1

1. Source Code->Compile->Execute process
2. Starting C program
3. printf statement
    1. Syntax
    2. Special characters, escape sequence

4. Comments in programs
5. Variables
    - Type and Name
    - Declaration
    - Assignment



Printing variables using printf

```
int year = 2026;
printf("This course begins in %d.\n", year);
```

Type	printf placeholder
int	%d
char	%c
float	%f
double	%f

Volume of a box : printing expressions

- Program to evaluate volume of a box and print it

```
#include <stdio.h>

int main(void)
{
    int length, width, height; //integer variables declared
    int volume;
    length=10;
    width=20;
    height=10;

    volume = length*width*height;

    printf("Volume = %d\n", volume);
    return 0;
}
```

- Can use expressions directly within printf

```
printf(“Sum = %d \n”, a+b);
```

- How to reuse for a separate box instance?

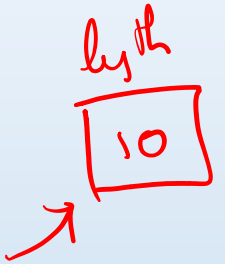
Reading Input

The scanf() function

Reusing programs

- Input: obtain the values needed by the program from user when program is running.
- Computation: evaluate expressions using those values.
- Output: report the result to the user.
- A new run can use different input without changing the source code.

The scanf function




```
int length;

scanf("%d", &length);

printf("You entered %d\n", length);
```

%d: expect an integer

length: the variable that will receive it

&: give scanf the location of that variable

Scanf waits for user input, consumes the characters appropriately from the input typed.

Caution: Output buffering

We may want to show a message/prompt for the user

Keep in mind “output buffering”: The message prompt may not show up always before the scanf, but is stored in a buffer to be printed later.

May result in varying behaviour across systems
Solution: instruct to flush the buffer immediately

```
int length;

printf("Enter length: ");
scanf("%d", &length);

printf("You entered %d\n", length);
```

To make printf print on terminal immediately

Either:

a) Write the following single statement in main():

```
setvbuf(stdout, NULL, _IONBF, 0);
```

OR

b) After every printf that needs immediate display, write:

```
fflush(stdout);
```

Making the cube program interactive

```
#include <stdio.h>

int main(void)
{
    int length, width, height;

    printf("Enter three dimensions: ");
    scanf("%d %d %d",
          &length, &width, &height);

    int volume = length * width * height;
    printf("Volume = %d\n", volume);
    return 0;
}
```

Multiple inputs in a single statement

```
int length, width, height;

scanf("%d %d %d", &length, &width, &height);

/* Input:  4 3 2
   Stores: length = 4
           width  = 3
           height = 2 */
```



Common starter error

```
int length;

/* Correct: scanf receives where length is stored. */
scanf("%d", &length);

/* Incorrect: do not write this. */
scanf("%d", length);
```

The format specifier must match the variable type

C type	scanf	printf	Typical use
int	%d	%d	whole numbers
char	%c	%c	one character
float	%f	%f	about 7 digits precision
double	%lf	%f	about 15–16 digits precision

Important: double uses %lf in scanf, but %f in printf.

Choose a type that matches the kind of value

- Use whole-number variables for counts, dimensions, and digit problems.
- Use floating-point variables when a fractional part matters.
- Use a character variable for one symbol, letter, or digit character.
- The variable’s type determines how its stored bits are interpreted.

Area of a circle

```
#include <stdio.h>

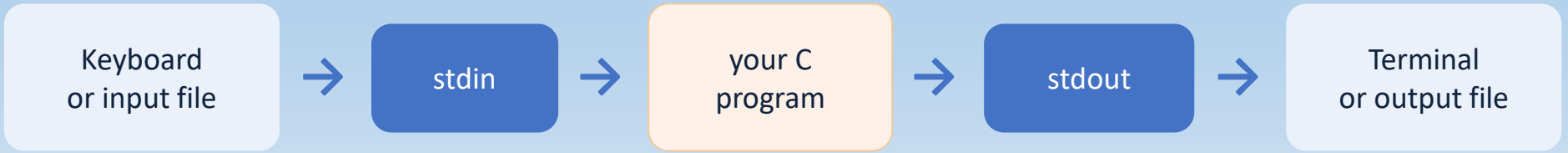
int main(void)
{
    double radius, area;

    printf("Enter radius: ");
    scanf("%lf", &radius);

    area = 3.14159 * radius * radius;
    printf("Area = %.2f\n", area);
    return 0;
}
```

Standard Input/Output Streams

- Standard input is the program's default incoming stream.
- Standard output is the program's default outgoing stream.
- In the terminal, the keyboard normally supplies standard input.
- In the terminal, standard output normally appears on the screen.
- Output buffering: things to be printed are stored in stdout, but not printed immediately to the terminal



Caution: character input

```
char grade;
int num;

scanf(“%d”,&num);
printf("Enter a grade letter: ");
scanf(" %c", &grade);

printf("Grade entered: %c\n", grade);

/* A space before %c skips earlier whitespace. */
```

Q: How do we store *strings* (i.e. full words and not just a single character) ? → More in Week 4/5

Assignment and Updates

Variables hold a current value that can change

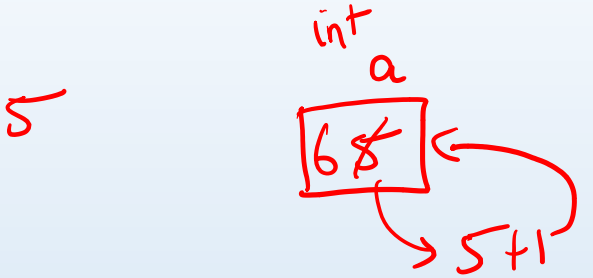
Recap: Declaration, initialization and assignment

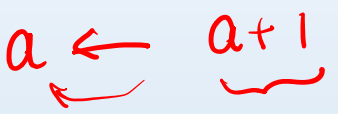
```
int score;          /* declaration: create the variable */

score = 10;         /* assignment: store 10 */

int lives = 3;      /* initialization: declare and give
                       an initial value together */
```

Updating a variable





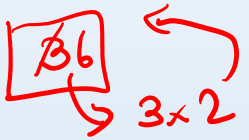

```
int a = 5;
a = a + 1;

/*Will the above compile?*/
```

How is the above done?

Step 1: read the old value of a -> 5
Step 2: evaluate the right-hand side -> 5 + 1 = 6
Step 3: store the result back in a -> a is now 6 */

What happens at each step




```
int x = 3;

x = x * 2;
x = x + 5;

printf("%d\n", x);
```

value of x = 6



Variable Updates

```
Statement              x before     x after
-----------------------------------------
int x = 3;                –            3
x = x * 2;                3            6
x = x + 5;                6           11

Only the current value remains stored in x.
```

More examples

```
/* Version A */
int a = 10;
a = a + 2;
a = a * 3;
```

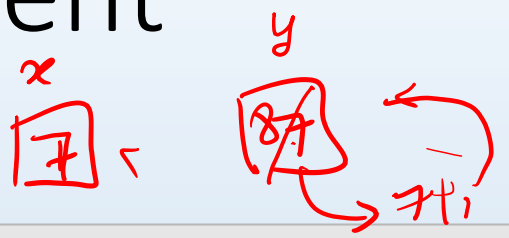
a stores 36

```
 /* Version B */
int b = 10;
b = b * 3;
b = b + 2;
```

b is 32

What is the value of a,b at the end in the above?

More on assignment



```
int x = 4;
int y = 7;

x = y;          /* copy y's current value into x */
y = y + 1;

printf("%d %d\n", x, y);   /* prints 7 8 */
```

Multiple Updates

```
int balance = 100;

balance = balance + 50;   /* deposit */
printf("After deposit: %d\n", balance);

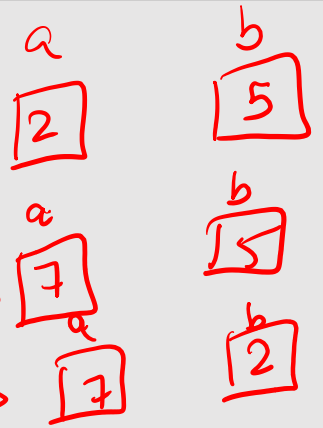
balance = balance - 30;   /* purchase */
printf("After purchase: %d\n", balance);
```

Predict the final values

```
int a = 2;
int b = 5;

a = a + b;
b = a - b;

printf("a = %d, b = %d\n", a, b);
```



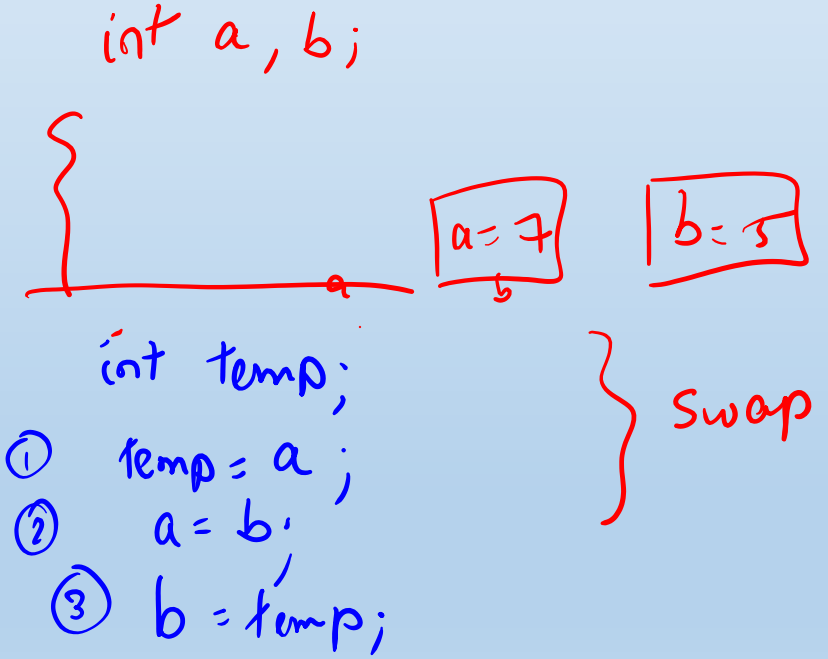
What does the program print?

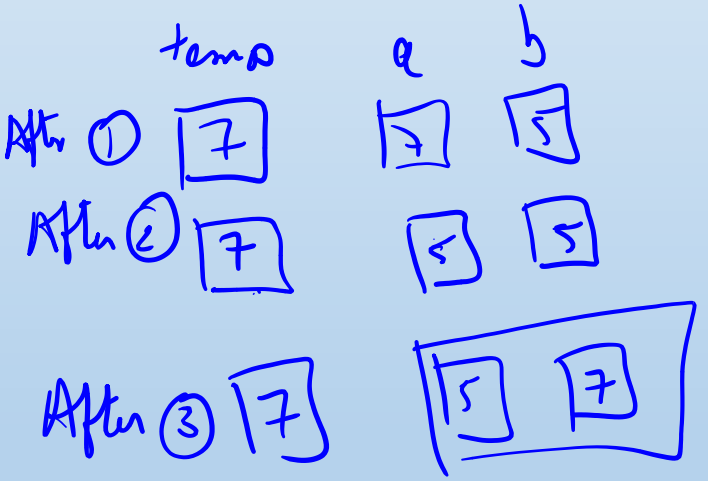
7 2

Write a program to swap two variables

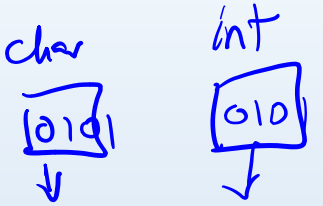
a, b : variables

Goal : program that swaps them



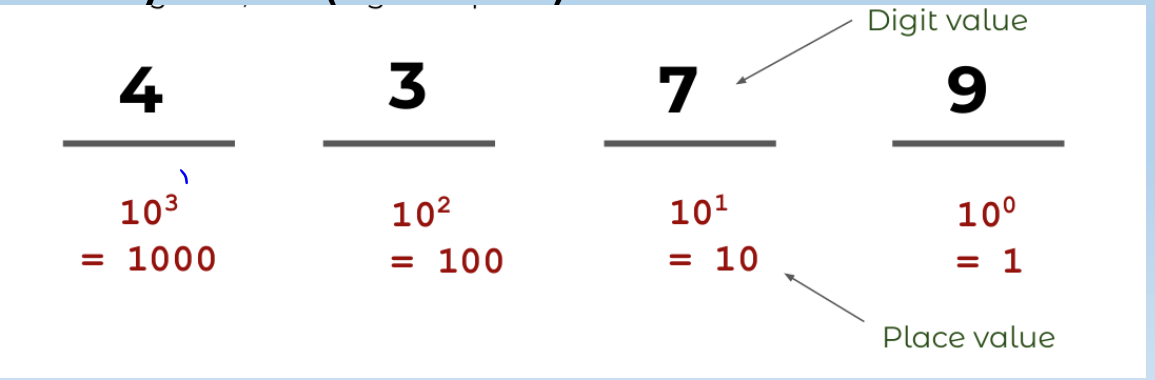


Representing variables in computer memory



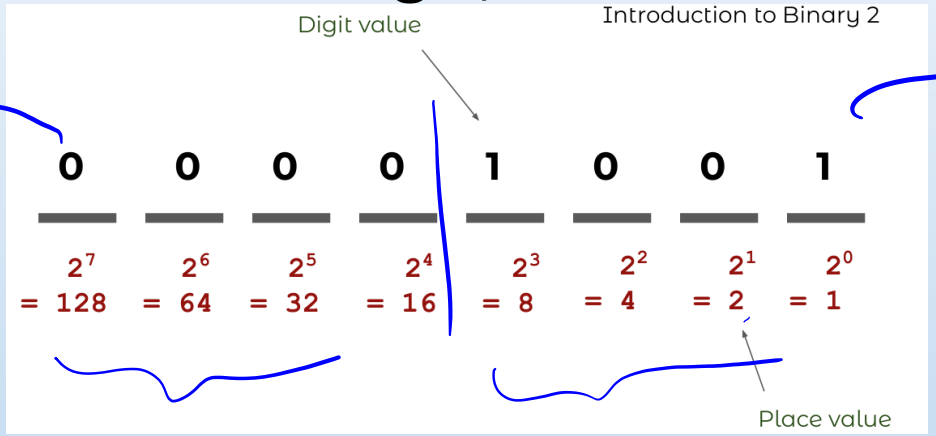
- In lab 2, you explored how char variables can be added to an integer:
  - `a+1 = b, a+2=c, …`
  - Why is this the case?

- Integers are stored as a sequence of 0's and 1's (binary form):
  - Decimal System (base 10)



Binary system

- Binary is the same logic, but to the base 2



most significant bit (msb)

least significant bit (lsb)

may representable number

$= 1\times2^7 + \dots \times 1\times2^2 + 1\times2^1 + 1\times2^0$

$= 2^8 - 1$

- *Hex*adecimal : base 16 (Digits are 0,…,9,A,B,C,D,E,F)

3A9F = 15,007

10, 11, 12 … 15

$3\times16^3 + A\times16^2 + 9\times16^1 + F\times16^0$

  - Usually when used, prefixed with a 0x. ,e.g. 0x3A9F

0011 = 3

- 11010011 in hexadecimal is: 3

Negative integers

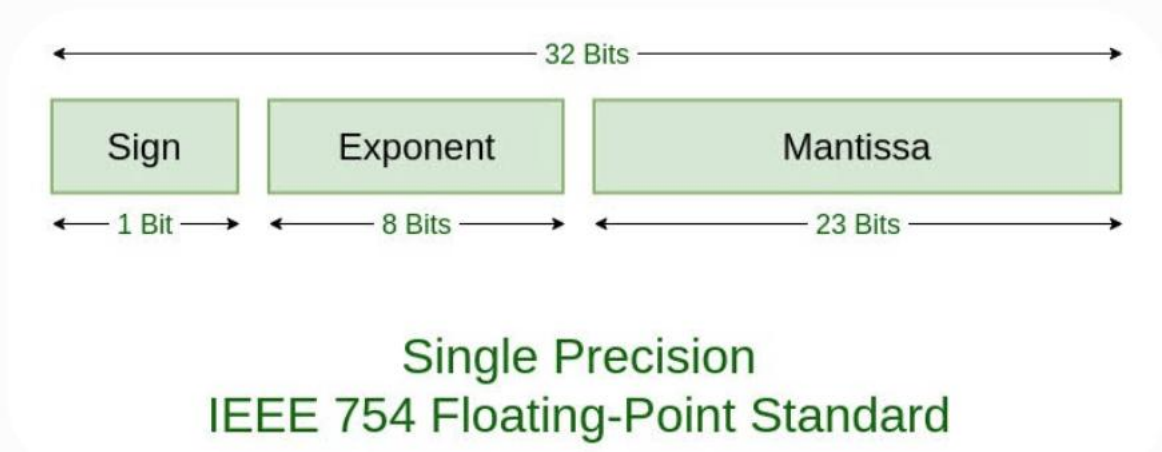
- **Negative Numbers:** Stored using a system called **Two's Complement**. To get a negative number, the computer flips all the bits of the positive version and adds 1.

- *Example:* To store -5, the computer takes 5 (00000101), flips the bits (11111010), and adds 1, resulting in 11111011.
  - This clever trick allows the CPU to use the exact same circuitry for addition and subtraction. (check why)

Characters

- In C, stored as a 1byte (8-bit) number
- The letter 'A' is stored as the decimal integer 65 (binary 0100 0001). The letter 'a' is stored as 97.
- When you tell C to print a char, it looks up that integer in an encoding table (usually ASCII) and prints the corresponding symbol.

Float and double



- Double has 8 bytes = 64 bits (1+11+52).
- Use double as your default for decimal numbers (it is the native floating-point type for C and most modern CPUs).
  - Float is used only in cases when space is to be saved (32 bits=4 bytes per number)
- Be careful about precision errors!

Arithmetic Expressions

Precedence order

```
a = b * b + 4;
```

Operations: * / % + -

- Similar to BODMAS rules for math
- Parenthesized expressions are evaluated first.
- Multiplication, division, and **remainder** come next.
- Addition and subtraction come after them.
- Operators at the same level are normally evaluated left to right.

```
int a = 2 + 3 * 4;          /* 14 */
int b = (2 + 3) * 4;        /* 20 */
```

Remainder (modulo) operation

```
int q = 17 / 5;
int r = 17 % 5;      /* remainder: 2 */


What will q have?
```

q has 3, r has 2

Splitting time

Split minutes into hours and minutes

```
int total_minutes;
int hours, minutes;

scanf("%d", &total_minutes);

hours = total_minutes / 60;
minutes = total_minutes % 60;

printf("%d hours %d minutes\n", hours, minutes);
```

Integer operations at the same level run left to right

```
int a = 25;
int b = 10;

int x = (2 * a) / b;
int y = (a / b) * 2;     /*  2 * 2  -> 4 */

The same numbers and operators can produce different results.
```

Ans 1: 50 / 10 -> 5 = x

Ans 2: 2 * 2 -> 4 = y

Floating point: Caution!

```
double average1 = (10 + 11) / 2;      /* Ans=10.0 */
double average2 = (10 + 11) / 2.0;    /* Ans=10.5 */

/* The division happens before the result is stored.
   Storing into double cannot restore a discarded
fraction. */
```

$\frac{(10+11)}{2} = 10$

Casting

```
int total = 21;
int count = 2;

double average;
average = (double) total / count;

printf("Average = %.1f\n", average);   /* 10.5 */

The stored value of total is still an integer.
```

(total * 1.0) / count

The integer total is *cast* as a double. Then divided by count. The result is a double value.

Making Decisions

Run statements only when their conditions are satisfied

Relational and logical operators

- Often, we would want to compare two values, and execute steps based on outcome of the comparison.

- A comparison is a *relational* operator, its result is a TRUE or a FALSE
  - `x > y`
  - `x == 0        /* comparison: is x equal to zero? */`

- We can take steps based on the outcome of a comparison using *conditional statements*.

```
Logical Equality:

number1==number2
    Evaluates to 1 (True) if number1 is equal to number2
    Evaluates to 0 (False) if number1 is not equal to number2
```

More comparison operators

Examples:

5 < 8, True

5 == 8, and !(5 == 8)

False

5 != 8

8 < 5 False

!(5 < 8) = False

Comparison/Relational		Combination (logical)	
<	less than	&&	and
<=	less than or equal	||	or
>	greater than	!	not
>=	greater than or equal		
==	equal to		
!=	not equal to		

Find the results

- `int a = 5, b = 8, c = 5;`
- `char grade = 'B';`

b - a > c	F
a < b && b < 10	T
!(a < c)	T
grade == 'B'	T
grade != 'A' && a >= 5	T
(a < b) && !(c != a)	T

Boolean values

- In C: typically represented as an integer
  - 0 = False
  - Any nonzero value = true

- For clarity, include stdbool.h, and use bool :

```c
#include <stdio.h>
#include <stdbool.h> // Must include this in C99/C11/C17

int main() {
    bool is_raining = true; // 'true' evaluates to 1

    if (is_raining) {
        printf("Take an umbrella!\n");
    }

    // Under the hood, a bool is still fundamentally
    // stored as an integer (0 or 1) in memory.
    printf("True is stored as: %d\n", is_raining);

    return 0;
}
```

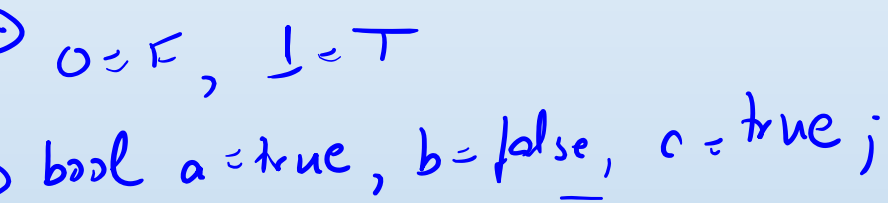
Precedence

- Use parenthesis when appropriate to avoid confusion

```
int a = 1, b = 0, c = 1;

// You write this:
if (a || b && c)

// C reads it as this:
if (a || (b && c))
```



Precedence	Operator	Meaning	Associativity (Tie-breaker)
1 (Highest)	!	Logical NOT	Right-to-Left
2	&&	Logical AND	Left-to-Right
3 (Lowest)	||	Logical OR	Left-to-Right

Conditionals

The if statement

```
x = 0;              /* assignment: store zero */
if (x == 0)
{
    printf("x is zero\n");
}
```

logical expn

what to do if expn is true

If block

```
int num;
scanf("%d", &num);

if (num < 0)
{
    num = -num;
}

printf("Absolute value = %d\n", num);
```

Logical operators

```
int age;
scanf("%d", &age);

if (age >= 13 && age <= 19)
{
    printf("Teenage range\n");
}

/* Both comparisons must be true. */
```

If-Else

```
int n;
scanf("%d", &n);

if (n % 2 == 0)
{
    printf("Even\n");
}
else
{
    printf("Odd\n");
}
```